

線上學習在半導體製造中的應用與資料漂移處理

背景：資料漂移與模型老化問題

在機器學習部署於製造業（尤其半導體製程）時，經常面臨資料漂移（Data Drift）或概念漂移（Concept Drift）的挑戰¹。也就是隨著時間推移，製程環境、設備狀態或材料特性可能改變，導致資料分佈或特徵與目標的關係發生變化¹。此時以過去靜態資料訓練的模型會逐漸失去準確性。如果不加以處理，模型效能將下降，嚴重時模型預測可能完全失效²。

在半導體製造中，**虛擬量測（Virtual Metrology, VM）**是常見應用，例如利用機台感測數據即時預測製程結果品質。但研究發現VM模型容易受到漂移影響，如製程條件或環境突然改變就屬**突發性漂移**，而設備逐漸老化則屬**緩慢漂移**^{3 4}。例如一項研究針對化學機械研磨（CMP）的VM資料，模擬了概念漂移情境，結果發現若直接使用固定歷史資料訓練的模型，其預測效能會隨漂移發生而顯著降低³。因此，需要**線上學習**方法來讓模型隨資料演進持續更新，以克服漂移、維持性能。

半導體與製造業中的線上學習實務

在製造業中已有一些線上學習與漂移應對的研究與實務案例。例如：

- **概念漂移檢測與適應**：國立臺灣大學的一項研究構建了線上學習框架，包含**投票式漂移偵測器（VDD）**與前景/背景模型機制^{5 6}。當監測到輸入資料的統計特性改變時觸發**漂移預警**，並啟動背景模型訓練；一旦確定漂移發生，就以新訓練的背景模型替換舊模型的一部分⁶。在公開的CMP資料集上，該方法比單一模型取得更穩健的預測效果⁴。這證明透過**漂移偵測+模型更新**可以有效緩解半導體製程中的資料漂移問題⁴。
- **虛擬量測中的漂移適應**：2025年發表於IEEE Access的一篇論文提出針對半導體VM的突發性漂移偵測與模型自適應方法⁷。雖然細節未盡述（屬於付費內容），但可推測該研究採用了**主動偵測漂移**並快速調整預測模型，以處理生產條件驟變造成的預測不準確⁷。類似地，另一研究探討如何在微製造過程中「應對概念漂移」，透過持續監控模型誤差及資料分布，及時觸發模型重訓練⁸。

上述實例顯示，在製造情境下使用線上學習結合漂移偵測是可行且有效的。不僅**即時監控**模型預測誤差或資料統計，可提早發現漂移，還能透過**增量更新或重訓**模型來維持預測品質。

應對資料漂移的策略與方法

為了解決資料漂移導致的模型老化，一般有兩大類策略：

1. **被動適應（線上增量學習）**：模型隨資料流持續更新，不特別去檢測漂移，而是每來一筆新資料（或一批資料）就融入模型中訓練。這包括：
 - **增量訓練（Incremental Learning）**：不重新從頭訓練模型，而是在既有模型基礎上用新資料**局部更新**參數。例如對於決策樹或隨機森林，可通過在線樹算法（如Hoeffding Tree）逐漸增長或剪枝；對於梯度提升決策樹（GBDT）模型，則可**追加新的樹**或更新葉節點值等。增量學習的優點是模

型能快速吸收新資訊（降低更新成本），但若舊知識與新資料衝突，模型結構未改變可能無法充分表達新概念。

- **滑動視窗（Sliding Window）**：只保留最近一段時間的資料來訓練模型，捨棄過久的歷史數據。透過移動時間窗口，模型**逐步遺忘舊概念**、專注於近期資料分佈⁹。窗口大小的選擇很關鍵：**較小的窗口**對最新趨勢更敏感，適合偵測與適應**突發性漂移**；**較大的窗口**平滑短期波動，對**緩慢漸變**的漂移較穩健¹⁰。有研究比較不同窗口策略，建議針對**漸變漂移**使用大窗口累積更多資料，以避免過度頻繁更新；而面對**突然漂移**時，採用小窗口或即時清除舊資料可更快捨棄過時模式¹⁰。此外，也有**自適應視窗**方法（如ADWIN算法），能動態調整窗口大小來平衡偵測速度與可靠性¹¹。

2. 主動因應（漂移偵測 + 重訓練）：這類屬**偵測-反應**方案，步驟為「監控 → 偵測漂移 → 模型更新」¹²：
漂移偵測：持續監控模型預測誤差或資料分佈統計，如發現顯著變化即判定漂移。例如**ADWIN**檢視視窗內統計量變化、**DDM（Drift Detection Method）**監控錯誤率的累積分布、或以KL散度檢測特徵分布變化等⁶。近年也有使用模型解釋指標（如SHAP值）來偵測製造資料漂移的嘗試，因特徵對預測的貢獻模式改變也能反映概念漂移發生¹³。

- **模型更新策略**：一旦偵測到漂移，觸發相應對策。簡單做法是**全量重訓練**最新資料（或擴充資料集重新訓練）。較進階的是**維持多個候選模型**：例如ARF（Adaptive Random Forest）算法中，每棵樹內建一個漂移偵測器，當某棵樹偵測到漂移預警，便啟動對應的**背景樹**開始學習新資料；若預警升級為確定漂移，則用背景樹替換原樹，達到平滑過渡¹⁴¹⁵。這種**前景/背景模型**架構可以在不中斷服務的情況下更新模型。另一種常見模式是**模型集成與加權**：同時維持一組模型，各模型針對不同時期的資料訓練，當偵測到漂移時調整模型權重（例如降低舊模型權重，提升新模型的影響）。這在**再訓練成本高或需保留部分舊知識**時特別有用¹⁶。

綜上，滑動視窗與漂移偵測是常用的線上學習指南。**決定視窗大小**可依據歷史數據的漂移週期調校，或透過交叉驗證模擬不同窗口長度對模型表現的影響來選擇。若有明顯的漂移點，不妨在該點附近縮短窗口或直接重置模型。此外，建立**自動漂移監測**機制相當重要，因為人工難以及時察覺模型何時開始失效。透過上述方法，最終目標都是使模型能**迅速對環境變化做出反應**，從而**提升整體性能並延長模型壽命**⁴。

模型與工具選擇：效能與可解釋性的權衡

為實現上述線上學習框架，需要選擇合適的模型與工具。考量到**預測效能與解釋性**都很重要（尤其在製造領域需要知道模型決策依據），這裡評估幾種候選方案：

- **CatBoost（分類提升樹）**：一種由Yandex開發的梯度提升決策樹演算法。CatBoost以對**類別型特徵**的原生支持和高準確性著稱¹⁷。在增量學習方面，CatBoost自版本0.15起支持**增量訓練模式（CPU）**，允許將資料分塊逐批訓練模型¹⁸。這非常適用於資料量極大（無法一次載入記憶體）的情況，可將資料流按塊讀入並不斷更新模型¹⁸。例如，有開源項目結合CatBoost與Ray實現分散式的增量訓練與服務，證明可在保持模型精度的同時，隨資料流持續更新模型¹⁹¹⁸。**優點**：CatBoost模型精度高且對異質資料表現良好；可透過繼續追加樹的方式逐步學習新資料。**挑戰**：需要謹慎控制新增樹的數量，以防止模型過度龐大或過擬合舊資料（CatBoost暫無內建機制移除舊樹）。對於概念大幅改變的情況，完全依賴增量更新可能不足，可能仍需定期重新訓練或結合漂移偵測進行模型替換。
- **LightGBM（輕量梯度提升機）**：微軟開源的GBDT實作，以高速訓練和低內存著稱。LightGBM沒有**明確的online learning接口**，但可透過 `train()` 配合 `init_model` 參數來基於舊模型訓練額外迭代，或使用 `Booster.update()` 與 `Booster.refit()` 方法進行更新²⁰²¹。差異在於：`refit` 不增添樹結構，只更新現有葉節點的值（樹結構固定）²²；`update` 則在現有模型上再增訓一輪（會新增少量樹）²³；而 `train(init_model)` 可以按設定的迭代數加入更多樹²³。**權衡點**：使用 `refit` 雖可避免模型愈趨複雜（樹數不變），但若新資料量少或分布差異大，強行套用舊樹結構可能導致預測劇烈改變或性能不佳²¹；使用 `train(init_model)` 最靈活強大，可大量加入新樹學習新

模式，但要適當降低學習率與樹數以免新資料影響過猛或模型膨脹²⁴。LightGBM開發者也提醒，簡單利用 `refit` 在流資料中**不保證精度提升**，因為資料分布變化可能需要新的樹分裂特徵才能擷取²⁵（僅更新葉值無法處理新模式）。因此，在**資料漂移明顯**的情況，LightGBM或需要定期**重啟訓練**或結合滑動視窗限制資料範圍，以確保模型結構能隨概念改變而演進。

- **River（線上機器學習工具包）**：River是Python的資料流機器學習庫，由原先的Creme和scikit-multiflow合併而成，專門用於**持續學習與動態資料流**²⁶。它內建多種**線上演算法**（如線上回歸、Hoeffding Tree、Naive Bayes、聚類等），以及**漂移偵測器**（如ADWIN、DDM）和評估工具²⁶。River中的**Adaptive Random Forest (ARF)** 特別適合有漂移的場景：每棵樹配備ADWIN偵測器，當偵測到漂移跡象，會啟動一棵**背景樹**開始學習新資料並在漂移確認後取代舊樹^{14 15}。這實現了模型局部更新與替換，讓整體森林隨資料變化自適應。**優點**：River的所有模型都支援以**逐樣本更新**（API通常為 `learn_one` / `predict_one`），非常適合流式資料和實時應用，而且包含漂移偵測模組方便組合實現。**缺點**：River的線上演算法多為針對流資料設計，單筆訓練雖高效但單模型複雜度較低（如Hoeffding Tree相當於淺層樹逐漸長成），純性能有時可能不及離線訓練的強力模型（如大型GBDT）。在資料量極大、關係複雜時，River模型可能需要較多樹或較複雜組合才能媲美CatBoost/LightGBM的精度。
- **EBM/GAM（可解釋附加模型）**：**Explainable Boosting Machine (EBM)** 是微軟提出的可解釋模型，本質上屬於廣義加法模型（Generalized Additive Model, GAM）的擴充。傳統上，提升樹模型雖有高準確度但難以解釋，而EBM巧妙地使用**多組淺層決策樹的加總**來學習每個特徵對目標的影響曲線，既保有提升樹的預測力又能輸出**透明的特徵作用圖**^{27 28}。EBM在訓練時對每個特徵單獨訓練一系列小樹（以及少量特徵交互項的小樹），再將其轉化為每個特徵取值對預測的**貢獻表**^{28 29}。最終模型即為這些**貢獻函數**的加總，可直接解釋「某特徵值對結果的影響是提高還是降低了多少」。**優點**：在不少資料集上，EBM達到接近黑盒模型（如XGBoost）的精度，同時可視為“玻璃盒”模型方便解釋³⁰。**缺點**：由於目前EBM實作主要是離線批次訓練，**缺乏線上增量更新能力**，因此在資料漂移環境下需要定期重新訓練（例如每天以最新窗口資料重新訓練EBM）。另外，GAM類模型若資料關係隨時間劇變，其過去學習的特徵效應曲線可能失效，需要透過更新訓練來重塑曲線。儘管如此，在**要求強解釋性**的場景（如製程品質預測需要說明影響因子）下，EBM/GAM是值得考慮的架構，可作為主要模型或其他模型的**解釋替代（surrogate）**。例如，可以用CatBoost持續預測但同步用近期資料訓練一個GAM來解釋當前階段的特徵影響，兩者搭配既有效能又有解釋。

小結：若偏重**即時性與持續學習**，專為流資料設計的River（ARF等）或增量更新方案（CatBoost/LightGBM的partial fit）更適合；若偏重**全域精度**和利用大量歷史數據，可定期批次重訓CatBoost/LightGBM模型並用滑動窗口控制資料範圍。而在需要**模型解釋**時，可以考慮以EBM/GAM為核心模型（需有計算資源支持頻繁重訓），或將其作為輔助解釋工具。每種模型都有特定強項，實務中也可以**混合搭配**：例如「**快速輕量模型 + 漸進更新**」做即時預測，週期性以「**高精度模型 + 全量重訓**」校正，並用GAM繪製重要特徵趨勢供專家參考。關鍵是在系統中預留彈性，方便替換或引入不同模型。

系統架構與設計考量

要建立一個能適應資料漂移的線上學習系統，需要從**資料管道**、**模型更新**、**監控反饋**、**架構設計模式**等方面通盤考量。以下提供一套整體框架的設計指南：

1. **資料管道與前處理**：系統首先需要捕捉製程中的資料流。例如從感測器或設備日誌持續收集數據，並通過消息佇列或資料匯流排傳輸。可以實作一個 `DataStream` 類別，負責連接資料來源並批次（或逐筆）輸出新資料記錄。由於線上學習通常要求即時性，前處理應盡量簡化和自動化，包括數值標準化、缺失值補全或簡單特徵轉換。在**架構上**可採用**管線（Pipeline）模式**，將資料依序經過前處理元件後送入模型。例如可以建立一個 `Preprocessor` 對象，實現方法如 `transform(sample)`，使每筆新資料在進入模型前都經過一致的處理步驟。

2. **模型管理與更新策略**：這是系統的核心，負責根據選定的線上學習策略訓練或更新模型。我們可以使用**策略模式（Strategy Pattern）**封裝多種模型更新方案，例如：

3. **Strategy A: 增量更新** – 每收集到一定筆數的新資料就使用既有模型的 `partial_fit()` 或等效方法更新（適用於如River模型或帶 `update()` 方法的模型）。例如對LightGBM，可實現一個 `LightGBMUpdater` 策略，每次呼叫時執行 `booster.update()` 或 `train(init_model=...)` 追加一棵樹。^{23 21}
4. **Strategy B: 滑動視窗重訓** – 週期性（如每小時或每天）觸發一次完全重訓，使用過去固定窗口範圍的資料重新訓練模型。可以有一個 `RetrainFromScratch` 策略，內部調用選定模型庫重新fit。為加速訓練，可考慮只初始化模型或使用Warm Start（如上次模型參數作為初值）後重新訓練。
5. **Strategy C: 模型交替/集成** – 維持兩套模型：例如 `current_model` 負責當前預測，同時 `candidate_model` 在背景用新資料訓練。一旦候選模型表現優於當前（透過驗證集或A/B測試比較），就**切換**為新模型。這實現所謂**Champion-Challenger**模式。可設計一個 `ModelManager` 類，內含現役模型和候選模型，並提供 `evaluate_and_swap()` 方法來自動切換模型。

透過上述策略模式，系統可以根據漂移情況靈活切換更新機制。例如當監測到緩慢趨勢時用增量更新微調，遇突發劇變時啟動重訓策略。

1. **漂移監控與觸發機制**：建議獨立一個**漂移偵測模組**，持續評估資料與模型狀態。這可透過**觀察者模式（Observer Pattern）**實作：`DriftDetector` 對象註冊觀察模型的輸出和真實值流（或者直接觀察資料分布變化），一旦偵測到異常（如模型預測誤差顯著上升，或資料統計分布偏離基準），就向模型管理模組發送事件通知⁶。常用的漂移指標包括滑動窗口內的**平均誤差**、**分類正確率曲線**的拐點、特徵直方圖差異等。River庫中的現成偵測器如ADWIN可直接用於這部分，或可自訂統計檢定（例如雙樣本Kolmogorov-Smirnov檢定檢測特徵分布變化）。一旦Observer發出“drift detected”訊號，`ModelManager` 就可以根據預先定義的策略（如上Strategy B或C）進行對應動作，比如重訓模型或啟用背景模型替換¹⁴。

2. **模型評估與效能追蹤**：在線上學習系統中，需要**持續評估**模型表現。可在架構中引入一個 `Evaluator` 或監控進程，採用**逐批評估**的方式：每當新資料進來時，先用當前模型做預測，將預測結果與真實值記錄下來計算即時損失，然後再將該樣本餵給模型更新（這叫做Prequential Evaluation，先預測再學習）。如此可得到模型**隨時間的性能走勢**。`Evaluator`可以定期將指標（如準確率、MSE、精確率召回等）輸出日志，或寫入資料庫供前端讀取。這部分數據同樣提供給漂移偵測器判斷模型是否異常失準⁴。例如，如果發現最近100筆預測的平均絕對誤差超過歷史均值兩倍，就觸發漂移警告。

3. **系統整合與介面**：最後是將以上組件整合並提供易於使用的介面。可以考慮構建一個**Streamlit**應用作為前端介面。用戶可以通過介面上傳或連接資料流（例如上傳一份含時間序列的製程資料CSV以模擬流資料），並設置參數（如視窗大小、選擇使用的模型類型、漂移偵測靈敏度等）。系統後端則按上述架構啟動線上學習流程。**設計模式**上，前端與後端可採用**MVC架構**：模型和學習流程作為後端控制核心，Streamlit界面呈現結果。透過Streamlit，可以在介面上實時顯示：當前模型評估指標、漂移偵測狀態（例如以圖表標示出漂移點）、特徵重要度或解釋（若使用EBM/GAM模型則顯示每個特徵的貢獻曲線圖）。對於不同模型，前端也可提供不同的**解釋模板**：如CatBoost/LightGBM可接SHAP值長條圖，EBM則顯示每個特徵對預測的影響曲線圖。

4. **擴展與維護**：系統架構應具有擴充性。例如透過工廠模式（Factory）管理模型實例化：增加新模型類型時（如日後引入神經網路增量學習），只需在工廠中實作對應生成函數，不影響主流程代碼。日志和監控也很重要，需有機制紀錄每次模型更新、漂移事件發生時間和處理動作，以供日後分析調優。部署上可採用容器化，後端服務獨立，Streamlit作為Web服務，方便在實際生產環境中與現有基礎設施（資料庫、訊息總線）對接。

結論與建議

綜合而言，要在半導體製造或其他工業領域成功引入線上學習，需同時考慮**資料**、**模型**、**系統架構**三方面：- 在資料層面，建立可靠的**資料流管道**並做好特徵工程自動化，為模型更新提供高品質的連續資料來源。- 在模型層面，選取適合的演算法或結合多種模型：利用**增量提升樹**（如CatBoost/LightGBM）獲得高性能，同時融入**線上演算法**（如River的ARF）應對快速漂移，再加上**解釋性模型**（如EBM/GAM）提高透明度，形成一個**互補的模型組合**。- 在系統層面，遵循良好的**設計模式**和架構原則，模組化地實現資料接收、模型訓練、漂移監控、結果展示等部分。透過策略模式和觀察者模式實現自適應更新，使系統能在無需人工干預下平穩地處理資料漂移。最後，以使用者友好的界面（如Streamlit）將整個流程串聯，方便工程人員與領域專家上手操作與觀察模型表現。

透過以上措施，模型將能更好地克服生產環境中的資料漂移挑戰，**持續維持甚至提升預測性能**⁴。同時，結合可解釋性方法，讓模型決策對工程人員而言不再是黑盒。總之，一個設計良好的線上學習系統將具備**適應變化的彈性與解釋決策**的能力，為半導體製程優化與智慧製造決策提供強有力的支援。

參考來源： 本文內容綜合了近期研究和實務報告，包括概念漂移處理的學術論文⁶⁴、CatBoost與LightGBM官方及社群文檔¹⁸²¹、River線上學習庫說明¹⁴、以及對解釋性模型EBM的介紹²⁷等，以確保觀點有據可查。

¹ ² ¹² Concept drift in AI. What is Concept Drift? | by Anmol Panna | Medium

<https://medium.com/@anmolpanna/concept-drift-in-ai-94badcfef532>

³ ⁴ ⁵ ⁶ ¹¹ ¹⁶ 發展基於資料定義域分佈之線上學習框架以分析具概念漂移之數據 = On the Development of Domain Distribution-based Online Learning Framework for Concept Drifting Data Analytics | Airiti Library 華藝線上圖書館

<https://www.airitilibrary.com/Article/Detail/U0001-0102230809406008>

⁷ Latent Space Alignment for Robust Detection of IoT Botnet Attacks in ...

<https://www.sciencedirect.com/science/article/pii/S0950705125017873>

⁸ Sudden Concept Drift Detection and Adaption in Virtual Metrology ...

https://www.researchgate.net/publication/394990957_Sudden_Concept_Drift_Detection_and_Adaption_in_Virtual_Metrology_for_Semiconductor_Manufacturing

⁹ Importance of Data Drift Detection - Analytics Vidhya

<https://www.analyticsvidhya.com/blog/2021/10/mlops-and-the-importance-of-data-drift-detection/>

¹⁰ Comparison based analysis of window approach for concept drift ...

<https://link.springer.com/article/10.1007/s10489-024-05890-4>

¹³ Understanding and detecting different types of drift | by Ajay Verma

<https://medium.com/@ajayverma23/understanding-and-detecting-different-types-of-drift-20d720c4727a>

¹⁴ ¹⁵ AdaptiveRandomForestClassifier - River

<https://riverml.xyz/0.11.1/api/ensemble/AdaptiveRandomForestClassifier/>

¹⁷ ¹⁸ ¹⁹ GitHub - xRiskLab/catboost-incremental: Incremental learning with CatBoost and Ray for scalable training, tuning, and serving of large ML models

<https://github.com/xRiskLab/catboost-incremental>

²⁰ ²¹ ²² ²³ ²⁴ python - LightGBM: train() vs update() vs refit() - Stack Overflow

<https://stackoverflow.com/questions/73664093/lightgbm-train-vs-update-vs-refit>

25 How does refit work? • Issue #1473 • microsoft/LightGBM - GitHub

<https://github.com/microsoft/LightGBM/issues/1473>

26 jmlr.csail.mit.edu

<https://jmlr.csail.mit.edu/papers/volume22/20-1380/20-1380.pdf>

27 28 29 How Do Inherently Interpretable AI Models Work? — Explainable Boosting Machine | by Indraneel Dutta Baruah | Nerd For Tech | Medium

<https://medium.com/nerd-for-tech/how-do-inherently-interpretable-ai-models-work-explainable-boosting-machine-f9e3718b1a4>

30 Explainable Boosting Machines (EBMs) - GeeksforGeeks

<https://www.geeksforgeeks.org/machine-learning/explainable-boosting-machines-ebms/>